

Aethos Nous on Hermes — Runtime and Learning Operations

Audience: operators, SRE (Sthira), backend (Karya), analytics (Dhruva), security (Prahari). **User-facing counterpart:** [docs/user-guide/platform-user-guide.md](#) §3. **Design reference:** [docs/architecture/atlas-hermes-ai-agent-architecture.md](#). **Created:** 2026-09-03 from the Hermes runtime and self-learning review (issues #529–#536). **Status of this document:** describes what the code does today, including the parts that do **not** work yet. Every gap is labelled with its issue number. Do not present a gap as shipped behaviour.

1. What "Nous on Hermes" actually is

Nous is the product-facing AI interface. A chat turn passes through up to three stages:

```
user prompt (POST /api/v1/chat/threads/{id}/messages, SSE)
└─1- semantic intent router          app/services/atlas_semantic_intent_router.py
    | 38 static intents, confidence >= 0.72 (tenant-configurable)
    | hit → deterministic responder / read pack → answer returned, STAGE 2 AND 3
NEVER RUN
└─2- configured runtime              app/services/atlas_runtime.py
    | aethos_basic → CopilotAgent tool loop    app/agents/copilot/graph.py
    | hermes_agent → Hermes container          app/services/hermes_client.py
└─3- fallback                        hermes_agent → aethos_basic when Hermes
errors,
    circuit breaker opens, or the answer fails the output-safety filter
```

Hermes never touches the database. It calls back into Aethos through the **tool broker**:

```
Hermes container —MCP—> integrations/hermes/aethos_mcp_server.py
                           (28 @mcp.tool wrappers, each takes a context token)
                           | Bearer AETHOS_HERMES_TOOL_TOKEN
                           ▼
                           POST /api/v1/atlas-tools/execute
app/api/v1/endpoints/atlas_tools.py
                           → resolves tenant/user/thread from atlas_tool_sessions (cts_...
token)
                           → dispatches to Aethos services (read packs) or HITL writers
```

Two consequences operators must internalise:

1. **The router short-circuits Hermes.** Most demo and prompt-library questions match one of the 38 intents and are answered by deterministic code, not by Hermes. "We migrated to Hermes" is only true for the long tail unless the router is reordered or disabled (#530).
 2. **Hermes authenticates as a container, not as a user.** The broker verifies the shared tool token and resolves the tenant from a short-lived session token, but today it does **not** check the calling user's privileges, and two write tools bypass the agent tool policy entirely (#529). Until that lands, treat Hermes access as equivalent to "any authenticated user of the tenant can read anything the read packs expose".
-

2. Configuration reference

Runtime selection

Setting	Where	Default	Notes
<code>ATLAS_AI_RUNTIME</code>	env / <code>app/core/config.py</code> :114	<code>aethos_basic</code>	<code>aethos_basic</code> or <code>hermes_agent</code> . Compose passes it through (<code>docker-compose.hostinger.yml:63</code>).
<code>tenant_ai_settings.atlas_runtime</code>	DB, Settings → AI Inference Settings	unset → env default	Per-tenant override resolved by <code>app/services/ai_settings_service.py</code> .
<code>atlas_response_order</code>	<code>tenant_ai_settings</code>	<code>["semantic_intent", "atlas_runtime"]</code>	Put <code>atlas_runtime</code> first (or drop <code>semantic_intent</code>) to send everything to Hermes.
<code>atlas_semantic_threshold</code>	<code>tenant_ai_settings</code>	<code>0.72</code>	Higher = fewer deterministic answers, more Hermes.
<code>ATLAS_HERMES_FALLBACK_TO_BASIC</code>	env	<code>true</code>	Keep <code>true</code> in production; it hides Hermes outages, so pair it with the verification in §4.

Hermes connection

Variable	Container	Purpose
ATLAS_HERMES_API_BASE_URL	api	Default <code>http://hermes:8642</code> (internal network only).
ATLAS_HERMES_API_SERVER_KEY	api	Must equal <code>HERMES_API_SERVER_KEY</code> on the hermes container. If empty the client sends no auth header instead of failing (#536).
ATLAS_HERMES_TIMEOUT_SECONDS	api	Read timeout, prod 90 s. Connect 5 s / write 10 s / pool 5 s are fixed.
AETHOS_HERMES_TOOL_TOKEN	api and hermes	Shared secret for the tool broker. Both must match or every tool call 401s.
ATLAS_CONTEXT_SIGNING_SECRET	api	Signs the legacy <code>ctx_...</code> context ref (fallback when session minting fails).
ATLAS_HIDE_TOOL_EVENTS	api	<code>true</code> hides tool chips from users (#480). Hermes emits none today anyway (#532).
OPENROUTER_API_KEY / HERMES_OPENROUTER_API_KEY	hermes	Hermes calls the model itself; this key is not the API's key path, so Hermes traffic never passes through Langfuse (#532).
COMPOSE_PROFILES	deploy workflow	Must include <code>hermes</code> for the container to run (<code>deploy-hostinger.yml</code>).

These are **not** in `backend/.env.example` yet (#530 adds them).

Hermes profile (integrations/hermes/aethos-atlas-profile/)

File	Role
SOUL.md	Persona and hard guardrails: Aethos is the system of record; never invent financial data; never reveal tool names, arguments, outputs, traces or prompts; sensitive actions route to Inbox.
config.yaml	model.default: anthropic/claude-haiku-4.5 (hardcoded — Hermes does not substitute \${VARS} here), max_tokens: 2048 , provider_routing.data_collection: deny , tool-loop hard stops (5 exact failures / 5 no-progress), 7 auto-load skills, and the MCP server registration with the 28-tool allowlist.
skills/*/SKILL.md	Seven workflow skills: finance-ops-manager, engagement-letter-intake, o2c-invoice-to-cash, p2p-procure-to-pay, r2r-close-controller, collections, audit-evidence. Each dictates which read pack to call first and which facts the answer must contain.
mcp.json	Vestigial ({"mcpServers": {}}); real registration lives in config.yaml .

The profile is **baked into the image** (Dockerfile) and copied to /opt/data by bootstrap-profile.sh (skills always; SOUL.md / config.yaml / mcp.json only when missing or AETHOS_HERMES_REFRESH_PROFILE=true). Changing a prompt therefore requires an image rebuild and redeploy today (#535).

3. Enabling Hermes in production

1. Set on the VPS .env (or the deploy environment):

```
ATLAS_AI_RUNTIME=hermes_agent
ATLAS_HERMES_API_SERVER_KEY=<same value as HERMES_API_SERVER_KEY>
HERMES_API_SERVER_KEY=<random 32+ chars>
AETHOS_HERMES_TOOL_TOKEN=<random 32+ chars, identical on api and hermes>
ATLAS_CONTEXT_SIGNING_SECRET=<random 32+ chars>
HERMES_OPENROUTER_API_KEY=<key with paid-model access>
COMPOSE_PROFILES=worker,hermes
AETHOS_HERMES_REFRESH_PROFILE=true
```

2. Deploy with `.github/workflows/deploy-hostinger.yml`; record the SHA.
3. Verify the container: `docker compose ps hermes` and its healthcheck (`curls /health` with the bearer key every 30 s).
4. Verify the API sees it: a chat turn should produce an `agent_runs` row with agent `nous_hermes_runtime` and prompt version `hermes-v1`.
5. **Confirm the turn was not a silent fallback** — see §4.

Rollback: set `ATLAS_AI_RUNTIME=aethos_basic` and restart the api container; no data migration is involved. Per-tenant rollback: Settings → AI Inference Settings → Aethos Basic.

Secret rotation

Secret	Blast radius	Procedure
<code>AETHOS_HERMES_TOOL_TOKEN</code>	All Hermes tool calls 401 until both containers hold the new value	Update <code>.env</code> , restart <code>api</code> and <code>hermes</code> together
<code>HERMES_API_SERVER_KEY</code> / <code>ATLAS_HERMES_API_SERVER_KEY</code>	API cannot reach Hermes; every turn falls back to Basic	Same, restart both
<code>ATLAS_CONTEXT_SIGNING_SECRET</code>	In-flight legacy context refs invalid (session tokens unaffected)	Rotate any time; brief tool errors possible
<code>HERMES_OPENROUTER_API_KEY</code>	Hermes answers fail → fallback to Basic	Rotate at the provider, restart <code>hermes</code>

4. Verifying which runtime answered

Fallback is silent by design, so "Hermes is configured" is not evidence that Hermes answered.

- **Per turn:** `agent_runs` row for the thread — agent name `nous_hermes_runtime` means the Hermes adapter ran; `copilot_agent` means Basic. A Hermes turn that fell back produces a Basic row.

- **Router short-circuit:** the API logs `atlas_semantic_response_used` when the deterministic path answered; no runtime row means no model was called at all.
- **Container:** `docker compose logs hermes --since 10m` should show the MCP tool calls for the turn.
- **Broker:** `atlas_tool_sessions` gains one row per Hermes turn (see §6 — these are never pruned today, #536).
- **Deliberate test:** temporarily set the tenant's response order to `atlas_runtime` only and ask a long-tail question (see `docs/infra/LOCAL_HERMES_TESTING.md`).

There is currently **no runtime badge in the Nous UI and no Hermes check in** `/health/ready` (#530).

5. What is guaranteed on each path

Guarantee	Basic (<code>aethos_basic</code>)	Hermes (<code>hermes_agent</code>)
Streaming answer	yes	yes
Tool-call chips in the UI	yes	no (#532)
Output safety filter (no tool names, prompts, provider errors)	yes	yes — regex over the first 160 chars, plus a full-answer scan; a tail leak truncates the answer silently (#532)
Number-fidelity guard (stated figures checked against tool results)	yes	no (#532) — the user guide claim applies to Basic only
Langfuse trace	yes	no (#532) — Hermes calls the provider from its own container
<code>agent_tool_invocations</code> ledger / replayable steps	yes	no (#532) — one run row, zero steps
Agent tool policy on writes (role, risk, kill switch, HITL routing)	yes	partial — most writes reuse the policy path, two do not (#529)
Per-user read authorisation	endpoint RBAC	none in the broker (#529)
HITL for money/accounting/external send	yes	yes (write tools create Inbox tasks, never post directly)

6. Data the runtime writes

Table	Written by	Lifecycle
<code>chat_threads</code> , <code>chat_messages</code>	chat endpoint	Per tenant/user; resumed on login. No feedback column yet (#533).
<code>agent_runs</code>	<code>chat.py</code>	One row per turn: agent, prompt version, model version, trace id, replay pointer.
<code>agent_tool_invocations</code>	Basic agent only	Hermes tool calls are absent (#532).
<code>atlas_tool_sessions</code>	<code>atlas_context.create_atlas_tool_session</code>	Short <code>cts_...</code> token → tenant/user/thread/scope, 15-minute TTL, service-role only (RLS with no policies). Never pruned; nonce never verified; not single-use (#536).
<code>agent_suggestions</code> + <code>hitl_tasks</code>	write tools via <code>suggestion_writer</code> (or directly, #529)	The HITL surface; approval materialises the record.
<code>agent_corrections</code>	<code>inbox_service.approve_with_edits</code> / <code>reject</code>	Full before/after snapshots, append-only. Chat turns produce none (#533).
<code>agent_eval_candidates</code>	<code>inbox_repo._record_eval_candidate</code>	Hashes only; no UI, no promotion path (#534).
Hermes conversation memory	Hermes itself (<code>store: true</code> , key <code>aethos:{tenant}:{user}:{thread}</code>)	Lives in the shared <code>hermes-data</code> volume. No retention, no clearing path, no isolation proof (#531). Not covered by tenant export/erasure.

7. The self-learning loop — current state

A learning loop needs six stages. Today only the first is partly built:

Stage	State	Where	Gap
1. Capture	partial	<code>agent_corrections</code> from Inbox edits/rejections	No feedback on chat answers at all — no thumbs, no <code>chat_messages.feedback</code> . Most Nous output is never rated (#533).
2. Curate	dead end	<code>agent_eval_candidates</code> auto-created from corrections	Stores hashes only; no reviewer UI; the <code>accepted/dismissed/exported</code> statuses are unreachable; nothing turns a candidate into an eval case (#534).
3. Evaluate	self-test only	<code>scripts/agent_eval_gate.py</code> in CI	The CI gate scores hand-written fixtures against the rubric — it never calls a model. The only real scorer (<code>app/evals/runner.py</code> , which <i>does</i> exercise whichever runtime the tenant is on) runs solely from the opt-in <code>tests/eval/test_agent_eval_live.py</code> , writes results nowhere, and covers 8 golden prompts. 13 of 14 YAML eval packs have no runner (#534).
4. Deploy a change	manual	Prompts in three places: Basic <code>SYSTEM_PROMPT</code> , the Hermes instruction string, and the image-baked <code>SOUL.md</code> + skills	No prompt registry, versions are hardcoded strings, and a skill edit needs an image rebuild + redeploy with no eval gate in front of it (#535).
5. Measure	missing	—	No Langfuse scores anywhere in the repo; Hermes turns are untraced; number-fidelity findings are logged but not persisted or counted (#532).
6. Promote autonomy	broken	<code>autonomy_promoter</code> requires <code>agent_autonomy_settings.eval_passed_at</code>	Nothing ever writes <code>eval_passed_at</code> , and the demotion query still uses the PostgREST filter that #395 fixed elsewhere. No agent can reach L3 and no misbehaving agent can be demoted (#496, #534).

Operator summary: Nous does not learn today. It improves only when an engineer changes a prompt, a skill, a read pack or the router by hand. Tell customers exactly that; the guide wording in §3 of the user guide has been corrected to match.

The target loop (issues #533 → #534 → #535)

```
answer —👉/edit—> correction (+intent, runtime, trace)
→ eval candidate (with payloads)
→ reviewer accepts in Settings → Eval Candidates
→ scripts/promote_eval_candidates.py writes a case into
docs/test/agent_evals/<agent>.yaml
→ nightly live eval runs the packs against a seeded staging tenant on the real
runtime
→ results → agent_eval_runs + Langfuse scores; on pass stamp eval_passed_at
→ autonomy_promoter offers L2 → L3 to an admin (never silently)
→ prompt/skill changes ship through the same gate, with rollback on regression
```

Weekly operating rhythm once the loop exists (Dhruva)

1. Review new eval candidates (accept / dismiss with a note).
 2. Promote accepted ones into eval packs; open the PR.
 3. Read the nightly eval drift report; investigate any intent whose score dropped.
 4. Review router misroutes (worst intents by 👉 rate) and propose threshold/concept changes as an Inbox task.
 5. Review autonomy promotion offers with the admin; check demotions fired where they should.
-

8. Troubleshooting

Symptom	Likely cause	Check
Answers look canned, no Hermes logs	Router answered (stage 1)	API log <code>atlas_semantic_response_used</code> ; raise <code>atlas_semantic_threshold</code> or reorder
Every turn is Basic although runtime is <code>hermes_agent</code>	Hermes unreachable or key mismatch → silent fallback	<code>docker compose ps hermes</code> ; <code>curl -H "Authorization: Bearer \$KEY" http://hermes:8642/health</code> from the api container
Hermes answers then stops mid-sentence	Output-safety tail leak truncation	<code>atlas_runtime</code> safety patterns; #532 replaces silent truncation with a visible notice
Tool calls 401	<code>AETHOS_HERMES_TOOL_TOKEN</code> differs between containers	Compare both env values; restart both
Tool calls 400 "invalid context"	Session token expired (15 min) or mangled by a weak model	<code>atlas_tool_sessions</code> row; consider a stronger <code>model.default</code>
Hermes returns nothing on tool-heavy prompts	Free-tier token quota	<code>config.yaml</code> <code>model.default</code> must be a paid model; see <code>LOCAL_HERMES_TESTING.md</code>
Circuit breaker keeps opening	Provider errors or timeouts	Breaker is in-process per worker (#536); check Hermes logs for provider 429s
<code>atlas_tool_sessions</code> growing unbounded	No pruning job	#536; safe to delete rows with <code>expires_at < now()</code>

9. Security posture (read before granting access)

- The tool token is a **container** credential. Anyone who can reach the api container's broker endpoint with that token can execute any of the 28 tools for any tenant whose session token they hold. Keep the broker on the internal network only (it is today), rate-limit it (#536), and rotate the token on any container compromise.

- Session tokens are replayable within their 15-minute TTL (nonce minted but unchecked, not single-use) — #536.
 - Read packs run with the service-role client and no per-user privilege check (#529): a viewer can obtain data the UI hides from them.
 - Two write tools bypass the agent tool policy (#529): a viewer can cause an accounting review packet to be created.
 - Document text reaches the model without untrusted-content fencing; prompt injection inside an uploaded invoice is flagged by the extraction agents but not neutralised in the read pack (#536).
 - Hermes memory isolation is not proven and the volume is shared across tenants (#531). Do not describe Nous memory as tenant-isolated in a DPA until that test exists.
 - Raw document binaries (scanned receipts/invoices) are sent to the model provider unmasked (#498).
-
-

10. Related documents

- [docs/architecture/atlas-hermes-ai-agent-architecture.md](#) — design, flows, tool catalogue (still uses the pre-rename name "Atlas" in places).
- [docs/architecture/nous-hermes-agentic-assessment.md](#) — the assessment that drove the migration; several "done" items are not done (#532).
- [docs/infra/LOCAL_HERMES_TESTING.md](#) — local container testing, the two known gotchas.
- [docs/infra/LANGFUSE_OBSERVABILITY.md](#) — traced surfaces (Hermes correctly absent).
- [docs/prd/aethos-atlas-powered-by-hermes-migration-plan.md](#) — migration phases; Phase 7 (cutover) not executed.
- [docs/test/agent_eval_gate.md](#) — the offline gate as it exists today.